

Числові типи

До вбудованих числових типів Python належать цілі, дійсні та комплексні числа. Стандартна бібліотека Python містить також модулі для роботи з числами з фіксованою точністю (decimal) та раціональними дробами (fractions).

Цілі числа

Літерали цілих чисел (тип `int`) записуються послідовністю десяткових цифр, перед якими може стояти знак числа (додатні числа зазвичай записуються без знаку): 125, -13, +64, 0. Python дає змогу подавати числа у двійковій (починаються з префікса `0b`), вісімковій (`0o`) та шістнадцятковій (`0x`) системах числення. Наприклад, літерал числа 111 у різних системах числення записується так: 111, `0b1101111`, `0o157`, `0x6f`.

Python підтримує необмежену точність подання цілих чисел¹. Наприклад, результатом виразу `2**100000` (два у стотисячній степені) буде число, яке складається зі 30103 цифр.

Дійсні числа

Літерали дійсних чисел (тип `float`²) розпізнаються інтерпретатором за символом десяткової крапки і/або експоненти (символ "e" або "E"): 3.14, 2.0, 2. (те саме, що 2.0), .2 (0.2), 1.5e-2 (0.015), 4E3 (4000.0, а не 4000), 5.e+2 (500.0). Діапазон значень типу `float` — приблизно від -10308 до 10308 з 16-ма десятковими знаками.

Комплексні числа

Комплексні числа розпізнаються інтерпретатором за символом уявної одиниці ("`j`" або "`J`"), який ставиться в кінці уявної частини: `3+2j`, `3.-2.5J`, `6j`. Їх можна також створити за допомогою вбудованої функції `complex(real, imagine)`, аргументами якої є дійсна (`real`) та уявна (`imagine`) частини.

¹ Кількість цифр обмежується лише обсягом доступної пам'яті

² Насправді тип `float` — це так звані числа з рухомою (плаваючою) крапкою, які не завжди дорівнюють дійсному числу, яке вони репрезентують (точність наближення складає 15-16 десяткових знаків).

Незалежно від запису ($2+j$ чи $2.+1.j$) дійсна та уявна частини є дійсними числами (2.0 і 1.0 відповідно). Отримати окремо дійсну та уявну частину можна через атрибути `real` та `imag` комплексного числа:

```
print((2+j).real)      # 2.0
c = 2+j
print(c.imag)          # 1.0
```

Операції з числами

Використовуючи арифметичні оператори, над числами можна виконувати такі операції (у дужках вказано відповідний оператор): додавання (+), віднімання (-)³, множення (*), ділення (/), отримання цілої частини (//) та остачі (%) від ділення, піднесення до степеня (**) та ін.

Наведений нижче код демонструє застосування цих операторів над цілими числами:

```
a, b = 123, 10
print(-a, a + b, a - b, a * b, a / b)    # -123 133 113 1230 12.3
print(a ** b)                            # 792594609605189126649
print(a // b, a % b)                     # 12 3
```

Результатом ділення завжди буде дійсне число. Якщо хоча б одне з чисел у виразі дійсне, то результатом виразу також буде дійсне число:

```
print(123 / 3, 123. // 10)               # 41.0 12.0
```

У складених виразах пріоритетність операторів наступна: піднесення до степеня, зміна знаку, множення та усі види ділення, додавання та віднімання. Для зміни черговості операцій (групування підвиразів) використовуються круглі дужки:

```
print(3*-5**2)          # -75
print((3*-5)**2)        # 225
```

Вбудовані функції та методи

Для операцій над числами, крім розглянутих вище операторів, можна користуватися вбудованими функціями⁴. Розглянемо деякі з них.

³ Оператор “-” також використовується для зміни знаку на протилежний

⁴ Більше математичних функцій для роботи з числами можна отримати зі спеціальних модулів стандартної бібліотеки

Функції конвертування типів:

```
int(3.95)          # 3 (відтинання дробової частини дійсного числа)
float(3)           # 3.0 (перетворення цілого числа в дійсне)
int('3')          # 3

int('3.95')       # ValueError: invalid literal for int() with base 10: '3.95'
float('3.95')     # 3.95
float('3,95')     # ValueError: could not convert string to float: '3,95'
```

Функції переведення в інші системи числення (зауважте, що результатом виконання цих функцій є рядок (str), тобто, вони повертають лише символічний запис числа в іншій системі числення.):

```
bin(111)           # '0b1101111' (у двійкову)
oct(0b1101111)    # '0o157' (у вісімкову)
hex(0o157)         # '0x6f' (у шістнадцяткову)
```

Функція заокруглення round() приймає першим аргументом число, а другим (необов'язковим) — кількість десяткових знаків, до якої потрібно його заокруглити:

```
round(3.95)        # 4 (заокруглення до найближчого цілого)
round(3.95, 1)     # 4.0
round(1/3, 3)      # 0.333
```

Інші математичні функції:

```
abs(-2)            # 2 (повертає модуль числа, у тому числі комплексного)
pow(2, 5)          # 32 (підносить 2 до 5-ї степені)
```

Числові типи як об'єкти мають методи, властиві своєму типу. Наприклад, дійсні числа мають метод перетворення у раціональне число (повертає кортеж з двох значень — чисельника і знаменника):

```
4.5.as_integer_ratio() # (9, 2)
```

цілі — метод обчислення кількості бітів, необхідних для їх подання:

```
n = 2500
n.bit_length()      # 12
```

а комплексні — метод обчислення комплексно спряженого числа:

```
2+3j.conjugate()    # (2-3j)
```

